

Informe Técnico de Entrega: Proyecto Desapalabrados

Nombre: Daniel Cardona Arango

Fecha: Junio 2026

Versión: 1.0 Final

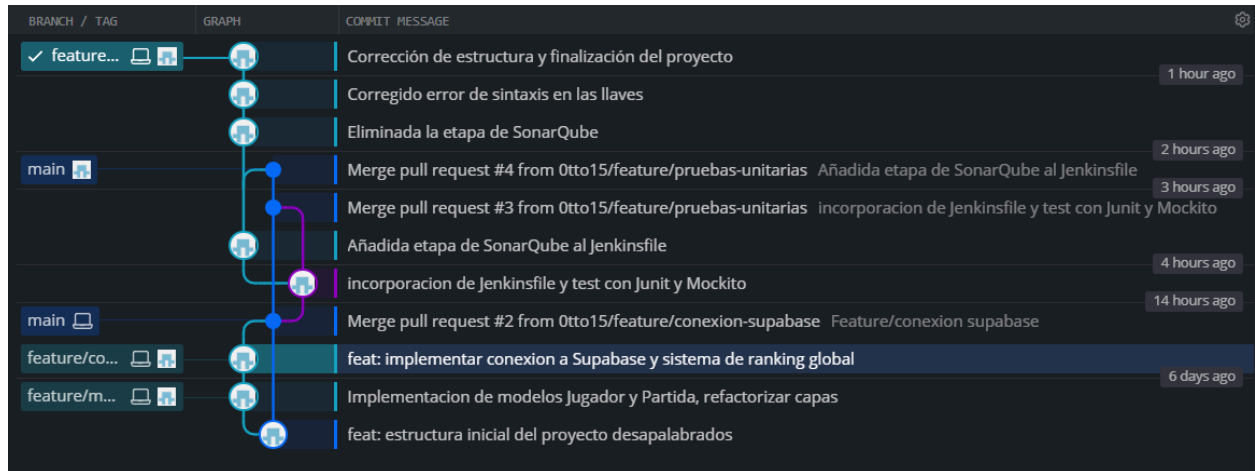
1. Introducción: Dominio del Problema y Alcance

El presente informe consolida el trabajo desarrollado por mi llamado "Desapalabrados". El dominio del problema consistía en construir una aplicación interactiva que retara la agilidad mental del usuario mediante la resolución de anagramas, implementando a su vez buenas prácticas profesionales de ingeniería de software.

Alcance Funcional y Ampliaciones: El proyecto no solo cumple con la funcionalidad mínima exigida (flujo de juego básico y ejecución en memoria), sino que se ha dotado de un gran valor añadido integrando una base de datos persistente en la nube (**Supabase**) para almacenar el *ranking* de jugadores de forma global, así como una interfaz de usuario pulida mediante **JavaFX**.

2. Repositorio Profesional y Construcción (Git & Maven)

Modelo de Ramas (Git): Se ha adoptado un flujo de trabajo donde la rama main está protegida. Todo el desarrollo de nuevas características (como la de integración de JUnit) se ha realizado en *feature branches* aisladas (ej. *feature/pruebas-unitarias*). Estas ramas se fusionaron a main a través de Pull/Merge Requests, asegurando la revisión previa del historial de commits.



Gestión con Maven: El archivo pom.xml ha sido estructurado y limpiado cuidadosamente, dividiendo las dependencias tecnológicas (JavaFX para UI, controlador JDBC para PostgreSQL) y de pruebas (JUnit 5, Mockito). Asimismo, se configuraron plugins esenciales para la ejecución de la cobertura, el empaquetado y el análisis estático de SonarQube, soportando perfiles de ejecución diferenciados (dev/test/prod) según el entorno.

3. Arquitectura y Diseño Orientado a Objetos (OO)

La solución se basa en una arquitectura de capas, aplicando patrones de abstracción robustos para reducir el acoplamiento.

Capa / Paquete	Responsabilidad Principal	Clases Relevantes
screens	Capa de presentación de la interfaz de usuario.	MainScreen
services	Lógica de negocio global y reglas de juego.	PalabraService
models	Entidades del dominio del problema.	Jugador, Partida
repository	Abstracción de datos para acceso en memoria y remoto.	IPalabraRepository, IRankingRepository

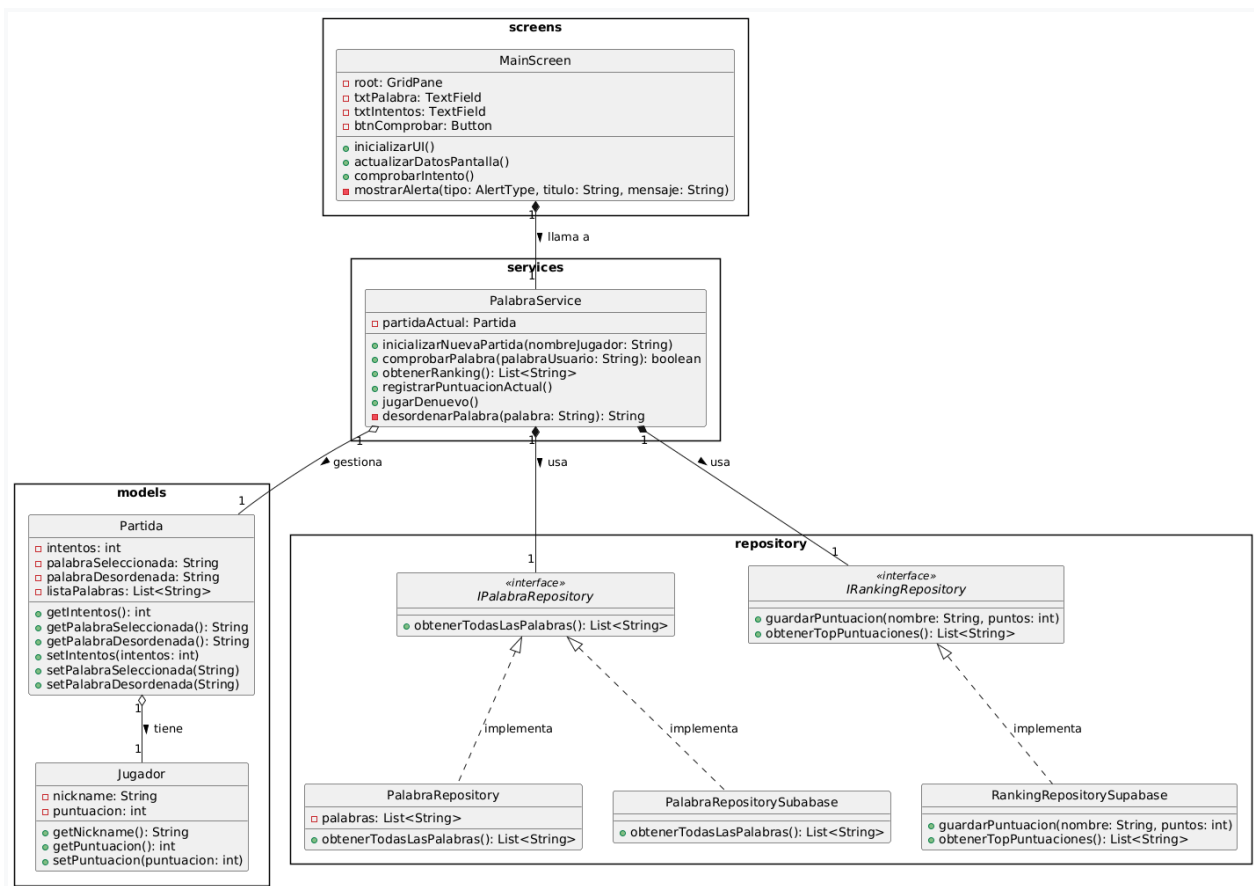
Principios SOLID y Polimorfismo:

El sistema cumple el principio de **Responsabilidad Única** delegando visuales a `MainScreen` y la lógica a `PalabraService`. Así mismo, se destaca el uso del principio de **Inversión de Dependencias** (mediante interfaces), logrando polimorfismo puro: la capa de servicio invoca a IPalabraRepository sin saber si por debajo se ejecuta PalabraRepository (lista local) o PalabraRepositorySupabase (redirección a base de datos externa).

4. Diagramas UML

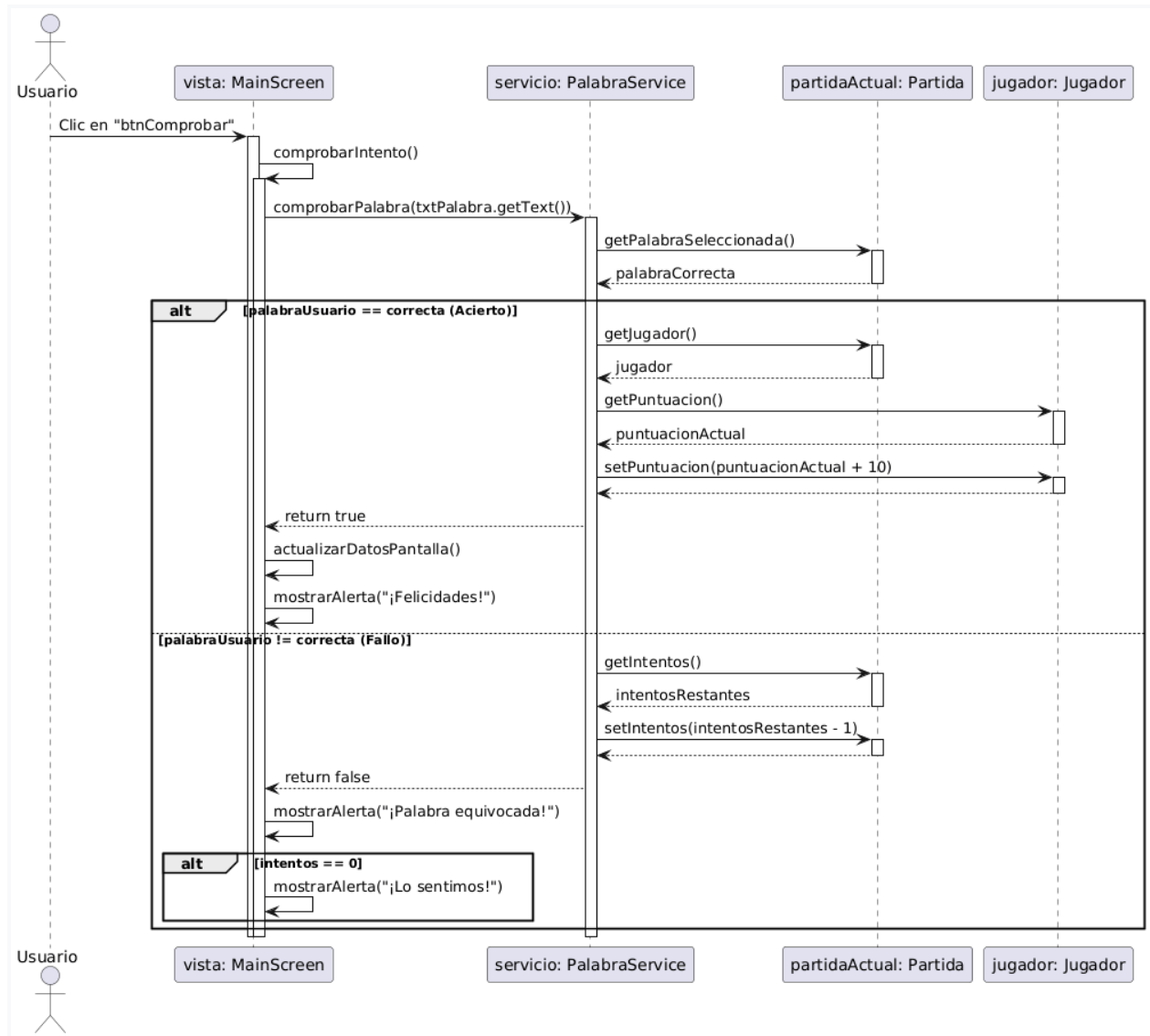
4.1. Diagrama de Clases (Estructura y Relaciones)

(Disponible también en la carpeta docs del repositorio en formato PDF).



4.2. Diagrama de Secuencia (Interacción de Objetos)

(Disponible también en la carpeta docs del repositorio en formato PDF).



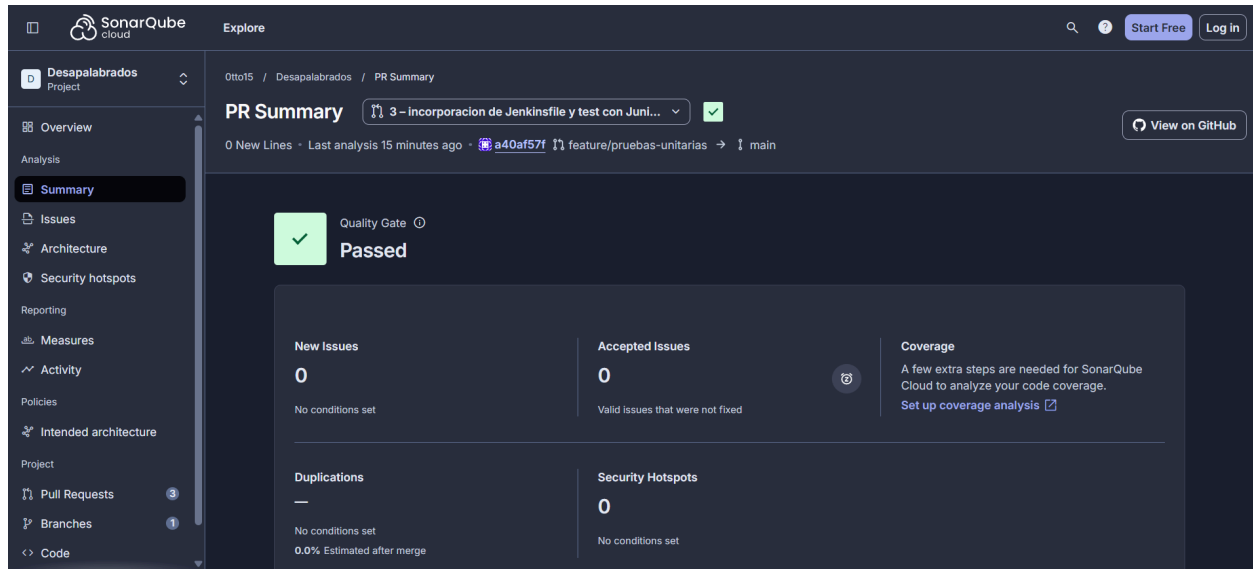
5. Testing: Estrategia, Cobertura y Mocks

La estrategia de testing está enfocada en el centro neurálgico del proyecto: la lógica de negocio (PalabraService). Debido a la arquitectura de inyección de dependencias, se han podido aislar las conexiones a Supabase utilizando **Mockito**. Al generar un *Mock* de `IPalabraRepository` y configurar `when(repositorio.obtenerTodasLasPalabras()).thenReturn(Arrays.asList("prueba"))`, se puede verificar que la lógica de comprobación de victoria y deducción de intentos de PalabraService es robusta y predecible.

6. Calidad del Código y Refactorizaciones Relevantes

Inicialmente, existía un acoplamiento donde los controladores gráficos gestionaban datos en bruto (*Code Smells*). La refactorización principal radicó en la extracción y delegación de toda esta lógica hacia clases puras de Java (*Services*). Adicionalmente, el análisis de código identificó vulnerabilidades y deuda técnica que se sanearon asegurando una gestión adecuada de las conexiones JDBC mediante bloques try-with-resources.

Métricas de Calidad Estática:



7. Pipeline CI/CD (Jenkins) y Lecciones Aprendidas

La integración continua ha sido automatizada a través de un servidor remoto de Jenkins en AWS. El archivo declarativo (Pipeline-as-code) implementa el siguiente ciclo estricto y secuencial:

1. **Checkout:** Sincronización del código fuente mediante clonación de Git.
2. **Build:** Limpieza y compilación del entorno usando mvn clean compile.
3. **Test:** Ejecución en servidor de la suite de pruebas mediante mvn test y reporte en formato JUnit XML.
4. **Package:** Generación del empaquetado optimizado de JavaFX (Artefacto .jar) ejecutando mvn package.
5. **Publish:** Acción de post-ejecución para publicar o archivar internamente el artefacto construido.

Jenkins

Desapalabrados-Pipeline

#4

Stages

Start

Checkout SCM

Tool Install

Limpieza y Preparación

Compilación

Ejecución de Tests

Post Actions

End

Search

Checkout SCM 0,18s

Tool Install 0,1s

Limpieza y Preparación 2s

Compilación 5s

Ejecución de Tests 6s

Post Actions 81ms

Post Actions 81ms Comenzó hace 1min Jenkins

Archivando los resultados de los tests... 12ms

Archive JUnit-formatted test results target/surefire-reports/*.xml 28ms

¡La pipeline se ejecutó correctamente! 19ms

¡La pipeline se ejecutó correctamente!

↑

↓

Jenkins 2.555.2

Jenkins

Desapalabrados-Pipeline

Stages

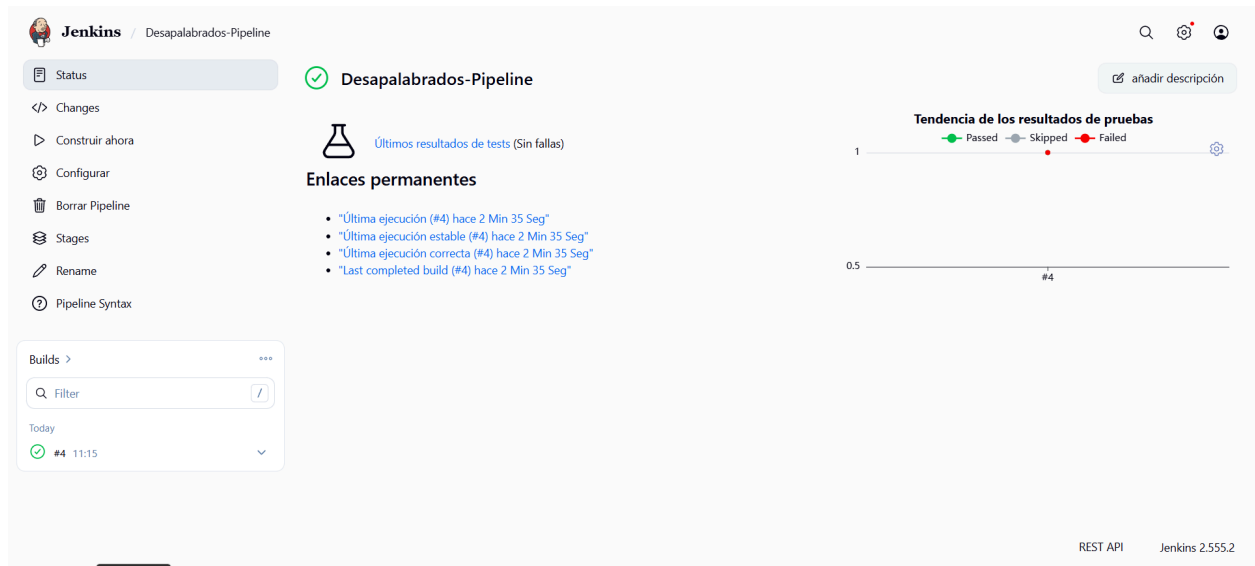
Build

7 de junio de 2026

#4 13:15 - 16s

○ - ● - ● - ● - ● - ● - ● - ○

Jenkins 2.555.2



Lecciones Aprendidas: La principal lección fue la gestión de configuraciones de entorno global en un servidor "limpio" de Linux (AWS). Resolver los conflictos entre las herramientas como jdk23 y la variable de entorno JAVA_HOME nos demostró el valor real de usar contenedores u orquestadores modernos y mantener los Jenkinsfiles altamente declarativos en lugar de depender de la configuración manual de las máquinas hospedadoras.